A PROJECT REPORT

ON

(EVENT MANAGEMENT SYSTEM)

SUBMITTED BY

KARAN KHARA

PARSHVA RAITHATHA

ACADEMIC YEAR 2026-27

T.Y- B.C.A SEM - 5

UNDER THE GUIDANCE OF

Mr. SHUNAK PUROHIT

SMT. C.Z.M GOSRANI B.C.A. COLLEGE

JAMNAGAR

SUBMITTED TO

SAURASHTRA UNIVERSITY

RAJKOT

24CS002UG04336
24CS002UG04277

A PROJECT REPORT

ON

(EVENT MANAGEMENT SYSTEM)

SUBMITTED BY

KARAN KHARA

PARSHVA RAITHATHA

ACADEMIC YEAR 2026-27

T.Y- B.C.A SEM - 5

UNDER THE GUIDANCE OF

Mr. SHUNAK PUROHIT

SMT. C.Z.M GOSRANI B.C.A. COLLEGE

JAMNAGAR

SUBMITTED TO

SAURASHTRA UNIVERSITY

RAJKOT

Elite EVENT

OSHWAL EDUCATION TRUST MANAGED
Smt. Chandramaniben Zaverchand Meghji Gosrani BCA College
Shah Bhagvandji Kachrabhai Edu. Complex, Ahead of Oshwal Circle, Indira Gandhi Marg, Jamnagar - 361004, Gujarat, India
Web: czmgbca.oshwaleducationtrust.org Email: info.czmgbca@oshwaleducationtrust.org
Tel: +912882573475 +912882563886

AFFILIATED TO SAURASHTRA UNIVERSITY, RAJKOT & APPROVED BY AICTE

CZMG BCA-135/2026-27/165 Date: 01/10/2026

CERTIFICATE OF PROJECT COMPLETION

This is to certify that the project entitled
"Event management system (web site)"
is a bonafide work carried out by
Khara Karan M (24CS002UG04277)
and **Raithatha Parshva P. (24CS002UG04336)**
as a partial fulfillment of the requirements
for the B.C.A. Semester 5 during the
Academic Year 2026-2027.

The project report meets the academic requirements for the project work prescribed by Saurashtra University, Rajkot.

EXAMINER NAME & SIGN	**PRINCIPAL SIGN**
Mr. Shaunak Purohit	/ /2026
PROJECT GUIDE	**EXAM DATE**

ABSTRACT

Elite Event is a web-based Event Management System developed for a college environment. It brings event information, student registration and event administration into a single website, so that students can find college events and register for them in one place.

Key features of the system include:

- Role-based login for Students, Teachers and Admin
- Event browsing with keyword search, category filter and sorting by date
- Event details page with Google Form based registration
- Recording of each student's registration, with a list of registered events on the student dashboard
- Event creation and editing by Teachers
- User management by the Admin: creating accounts, changing roles, suspending or reactivating and deleting users
- Admin view of all event registrations and management of the Instagram posts shown on the home page

The system is developed using HTML, CSS and JavaScript on the front end, PHP on the back end and MySQL as the database, and runs on an Apache server through XAMPP. Passwords are stored as hashes, forms are protected with CSRF tokens and every protected page is guarded by role.

ACKNOWLEDGEMENT

The feeling of gratefulness to any one's help directly arises from the bottom of heart. A small but an important and timely help can prove to be a milestone in one's life.

This project is dedicated to all the people, whom i met, took guidance, talked, interviewed and learned something from them. At this occasion i sincerely thank all of them while submitting this project report.

I would like to thank MR. Shaunak Purohit for giving time-to-time valuable guidance during the project and giving me guidance in taking my curriculum decision and choosing, initiating and getting on with the project.

I always have felt the invisible help from the almighty, without the blessings of almighty, I could not have succeeded.

PROJECT PROFILE

DEVELOPER	
Name: **(1) Karan Khara** **(2) Parshva Raithatha**	**Enrollment Number:** (1) 24CS002UG04277 (2) 24CS002UG04336
PROJECT DETAILS	
Project Title:	Event management system
Duration:	3 Months
Name of Project:	Elite Event
Platform:	PHP & CSS (Front End) JavaScript (Back End) Mysql (Database)
Team Size:	2
GUIDE INFORMATION	
Name of Guides	
MR. Shaunak Purohit	

INDEX

Sr No.	TITLE	Page No.
	TITLE PAGE	I
	ABSTRACT	II
	ACKNOWLEDEMENT	III
	PROJECT PROFILE	IV
	CHAPTERS	
1	SDLC OVERVIEW	7
2	REQUIERMENT GATHERING AND ANALYSIS	8
	2.1 Organization Details	
	2.2 Meetings	
	2.3 Data which will be Input into the system	
	2.4 Data which will be Output from the system	
	2.5 Type of Project	
	2.6 Method of collecting Data	
3	SYSTEM REQUIREMENT SPECIFICATIONS	4
	3.1 Introduction	
	3.2 Overall Description	
	3.3 External Interface Requirement	
	3.4 System Features	
	3.5 Other Nonfunctional Requirements	
	3.6 Feasibility Study	
4	SYSTEM ANALYSIS AND MODELING	11
	4.1 Use case Diagram	
	4.2 Normalization and E-R Diagram	
	4.3 Data Dictionary	
	4.4 Functional and Behavior Modeling	
	4.5 Gantt Chart	
5	TEST CASE	
6	SCREENSHOTS	38
7	LIMITATIONS AND FUTURE ENHANCENT	40
8	CONCLUSION	41
9	REFERENCES AND BIBLIOGRAPHY	51

1. SDLC OVERVIEW



2. REQUIREMENT GATHERING AND ANALYSIS

2.1 ORGANIZATION DETAILS:

1. Name of the Organization: -

Elite Event Solutions Pvt. Ltd.

2. Brief detail of the organization: -

Elite Event Solutions Pvt. Ltd. is a professional event management company specializing in organizing corporate events, cultural festivals, weddings, seminars, workshops, and conferences. The organization manages multiple events throughout the year and requires an online Event Management System to streamline event planning, registrations, bookings, scheduling, and participant management.

2.2 MEETINGS:

I. Meeting with Principal

1. Name of Principal

-Dr.RajeshKumar(Registrar)

2. Principal's requirement in her words.

The company requires a centralized Event Management System.

The system should allow administrators to create, edit, and manage events.

Users should be able to register, book events, and receive confirmations.

The system should maintain participant records and generate reports.

II. Meeting with user

1. **Name of the user.**

   Ms.NehaSharma

   (Administrative Officer)

2. **User requirement in her words.**

- The system should simplify event registration.
- Participants should be able to browse upcoming events.
- Booking confirmations should be generated automatically.
- Event details should be updated instantly.
- Reports should be available whenever required.

2.3 Data which will be the input into the system….

- Event Details
- Organizer Details
- Participant Information
- Venue Details
- Event Schedule
- Booking Details
- Payment Information
- Images
- PDF Documents
- Text

2.4 Data which will be the output from the system….

- Event List
- Booking Confirmation
- Participant Reports
- Payment Reports
- Event Schedule
- Registration Summary
- PDF Reports
- Notifications

2.5 Type of project….

- **Online Web-Based Event Management System**

2.6 Method for collecting data.

- Interviews with Project Manager
- Meetings with Event Coordination
- Requirement Analysis
- Observation of Existing College Event Management Process
- Study of Existing Event Management Systems

3. SYSTEM REQUIREMENT SPECIFICATIONS

3.1 Introduction:

3.1.1 Purpose:

A web-based platform designed to simplify event planning and management—allowing admins to create and manage events, and users to browse, register, book tickets, and manage bookings online..

3.1.2 Intended Audience and Reading Suggestions:

Serves as a requirement specification guide for event organizers, administrators, participants, developers, testers, and maintenance teams.

3.1.3 Product Scope:

An end-to-end automation tool enabling organizers to handle events, registrations, ticket bookings, announcements, and reports, while providing users with seamless online registration, booking, and event notifications.

3.2 Overall Description:

3.2.1 Product Function:

(i.) Administrator Module :

- Login
- Dashboard
- Create Event
- Update Event
- Delete Event
- Manage Event Categories
- Manage Users
- Approve Registrations
- Manage Bookings
- Publish Announcements
- Maintain Database

(ii.) User Module :

- Register
- Login
- Update Profile
- Browse Events
- View Event Details
- Register for Events
- Book Tickets
- View Booking History

3.2.2 User classes and characteristics:

(i.) User Classes :

- Administrator
- Teacher
- Registered User (Participant)

(ii.) User Characteristics :

(a) Administrator:

The administrator should be capable of managing events, user accounts, bookings, announcements, reports, and the system database efficiently.

(b) Teacher :

The Teacher should be capable of managing events, bookings, announcements, reports, and the system database efficiently

(c) Student :

The user should have basic knowledge of using a computer or smartphone and internet browsing. Users should be able to register, browse events, and complete event bookings online.

3.2.3 Operating Environment:

The Event Management System operates through the Internet and can be accessed using desktops, laptops, tablets, and smartphones.

Operating System

- Windows 10 / Windows 11
- Linux

3.2.4 Design and Implementation constraints:

- Graphical User Interface (GUI) is available in English.
- Internet connection is required.
- Users must register before booking an event.
- Only the administrator can manage events and system data.
- The system should support responsive web design

3.2.5 Assumptions and dependencies:

- Administrator account is already created.
- All users have internet access.
- Database server is available at all times.
- Roles and permissions are predefined.
- Users provide valid registration details

3.3 External Interface Requirement:

3.3.1 User Interface:

The system provides a simple, attractive, and user-friendly interface.

If the user is not registered, they can create a new account by filling in the registration form. After successful registration, users can log in using their credentials. If an invalid username or password is entered, an appropriate error message will be displayed.

The application includes:

- Home Page
- User Registration
- Login Page
- Event Listing
- Event Details
- Booking Page
- User Dashboard
- Admin Dashboard

3.3.2 Software Interface:

- **Operating System:** Windows 10/11, Linux, Android, iOS
- **Front End:** HTML5, CSS3, Bootstrap, JavaScript
- **Back End:** PHP
- **Database:** MySQL
- **Web Server:** Apache (XAMPP/WAMP)
- **Development Tool:** Visual Studio Code

System Features:

- **User Registration and Login**
- **Administrator Login**
- **Event Creation and Management**
- **Event Categories**

- **Online Event Registration**
- **Ticket Booking**
- **Event Gallery**
- **Booking Management**
- **User Profile Management**
- **Announcements and Notifications**
- **Report Generation**

3.4 Other Nonfunctional Requirements:

3.5.1 Performance requirement:

- **Fast Response:** Pages must load without noticeable delay under normal college traffic.
- **Input Validation:** Required fields and URLs are validated with clear error messages.
- **Optimized Queries:** Database operations rely on prepared statements and lightweight queries (unbenchmarked).

3.5.2 Security Requirement:

- **Role-Based Access:** Restricted admin/teacher routes.
- **CSRF Defense:** Token validation on forms.
- **SQLi Prevention:** PDO prepared statements.
- **XSS Prevention:** htmlspecialchars() output escaping.
- **Hidden Route:** Unlinked admin login.
- **Student Auth:** Logged-in registration only.

3.5.3 Safety Requirement:

- Event, user and registration records are stored in the database.
- One user and one event can have only one registration; the README defines a UNIQUE (event_id, user_id) rule and the code handles duplicate attempts.
- The Google Form URL is accepted only when it belongs to docs.google.com and contains /forms/.

- **Feasibility Study**

A feasibility study is a preliminary investigation to determine whether the proposed system can operate effectively and satisfy the identified requirements.

3.6.1 Operational Feasibility: -

The system replaces scattered event announcements and separate registration sheets with a single website. Students use simple pages for search and registration, and teachers use one form to publish an event. The workflow matches the way events are organised in a college, therefore the system is operationally feasible.

3.6.2 Technical Feasibility: -

The system is developed with PHP, HTML5, CSS3, JavaScript, a relational database and XAMPP. These technologies are widely available and well documented, and the development team has the skills to use them. Event registration data collection is handled by Google Forms, which removes the need to build a custom form builder. The project is therefore technically feasible.

3.6.3 Economic Feasibility: -

PHP, XAMPP and the database software are available free of charge, and Google Forms can be used without a licence cost for the required purpose. No paid library or payment service is used, so the development and running cost is low.

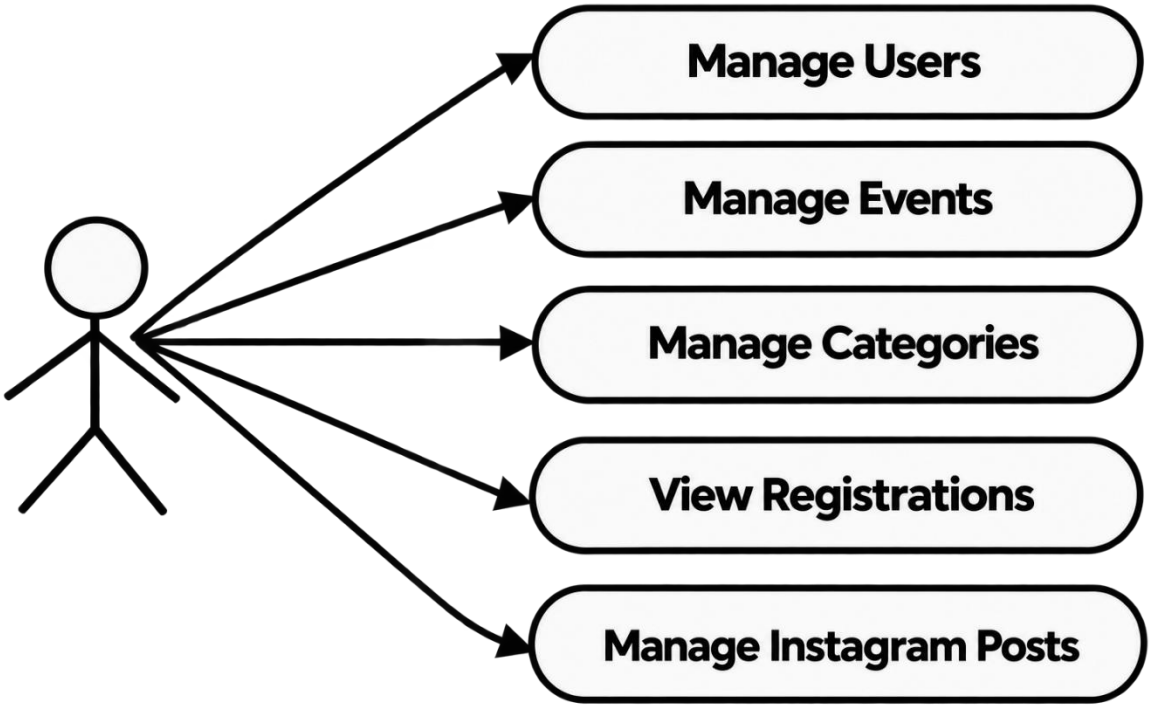
3.6.4 Schedule Feasibility: -

Because the technologies were familiar to the team and a part of the work (registration forms) is delegated to Google Forms, the planning, design, development and testing activities could be completed within the semester project schedule.

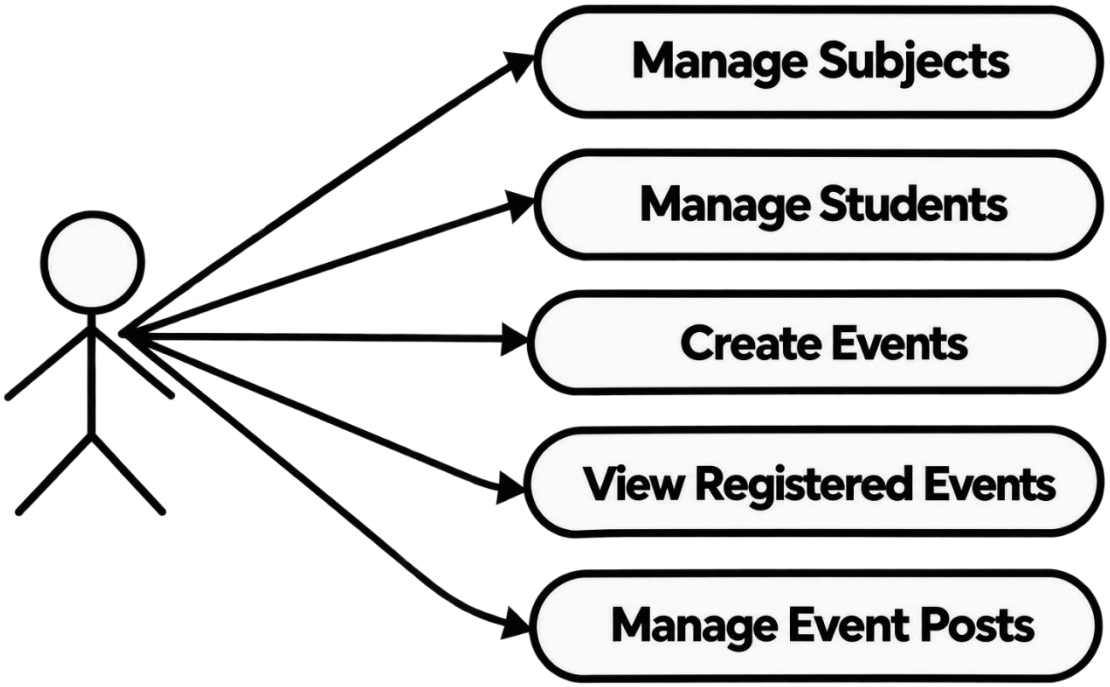
4. SYSTEM ANALYSIS AND MODELING

4.1 Use case Diagram:

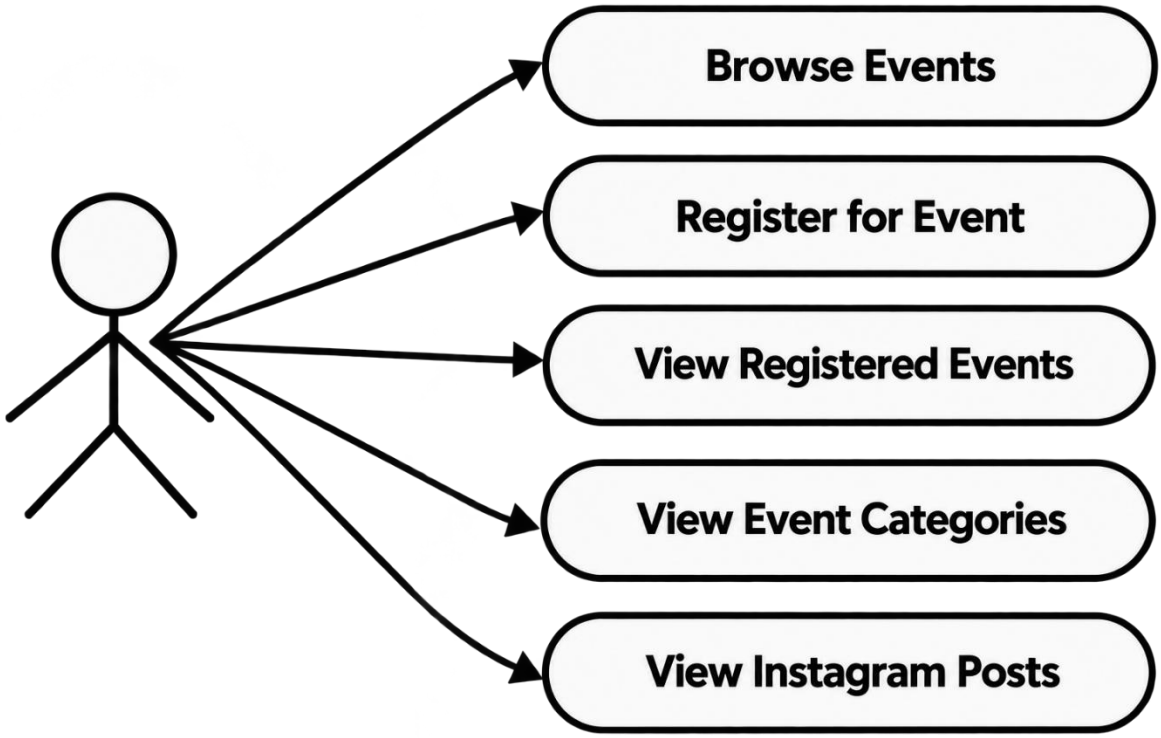
1) Admin Side :



2) Teacher Side :



2) Student Side :



4.2 Normalization & E-R Diagram:

Normalization is the process of organizing the columns (attributes) and tables (relations) of a relational database to reduce data redundancy and improve data integrity.

- **De Normalized Form:**

Registration ID	Event ID	Event Title	Category	Date	Time	Venue
1	6	Hackathon 3.0 — 24 Hours	tech	9/15/2026	9:00:00	CS Block Lab 3, GEC
2	3	Heritage Craft Market	market	9/2/2026	11:00:00	Central Park Grounds

City	Organizer	User ID	User Name	User Email	Role	Registration Date
jamnagar	IEEE Student Chapter	1	karan khara	karan2khara2@gmail.com	student	10/3/2026 16:44
Ahmedabad	Craft Guild of Gujarat	2	Parshiva	Parshiva@gmail.com	student	10/3/2026 16:48

➢ **Normalized Form:**

A. 1st Normal Form:

1) **Table Name :** users

Primary Key: id (Auto Increment)

id	name	email	password_hash	role	status	created_at
1	Site Admin	admin@eliteevent.local	*****	admin	active	8/8/2026 6:51
2	karan khara	karan2khara2@gmail.com	*****	student	active	8/8/2026 6:52
5	mayur	mayur@gmail.com	*****	teacher	active	10/2/2026 20:38

2) **Table Name :** events

Primary Key: id (Auto Increment)

id	title	category	date	time	venue	city
1	Startup Founders Meetup	meetup	9/5/2026	18:30:00	WeWork Galaxy	Hyderabad
2	Hackathon 3.0 — 24 Hours	tech	9/15/2026	9:00:00	CS Block Lab 3, GEC	Ahmedabad
3	Inter-Dept Cricket League	sports	9/25/2026	8:00:00	College Sports Ground	Ahmedabad

organizer	description	google_form_url	created_by	created_at	updated_at
StartupSphere HYD	Monthly meetup for startup founders and aspiring entrepreneurs. Pitch, network, and learn from fellow builders.	NULL	NULL	8/8/2026 17:26	8/8/2026 17:26
IEEE Student Chapter	Build innovative solutions in 24 hours. Open to all students.	NULL	NULL	8/8/2026 17:26	10/3/2026 16:52
Sports Committee, GEC	Annual inter-departmental cricket tournament. Register your team and compete for the championship trophy.	NULL	NULL	8/8/2026 17:26	8/8/2026 17:26

3) **Table Name : registration**

Primary Key: id (Auto Increment)

id	event_id	user_id	registered_at
1	6	2	10/3/2026 16:44
2	3	2	10/3/2026 16:48

B. 2nd Normal Form:

1) **Table Name :** users

Primary Key: id (Auto Increment)

id	name	email	password_hash	role	status	created_at
1	Site Admin	admin@eliteevent.local	*****	admin	active	8/8/2026 6:51
2	karan khara	karan2khara2@gmail.com	*****	student	active	8/8/2026 6:52
5	mayur	mayur@gmail.com	*****	teacher	active	10/2/2026 20:38

2) **Table Name :** events

Primary Key: id (Auto Increment)

Foreign Key: email

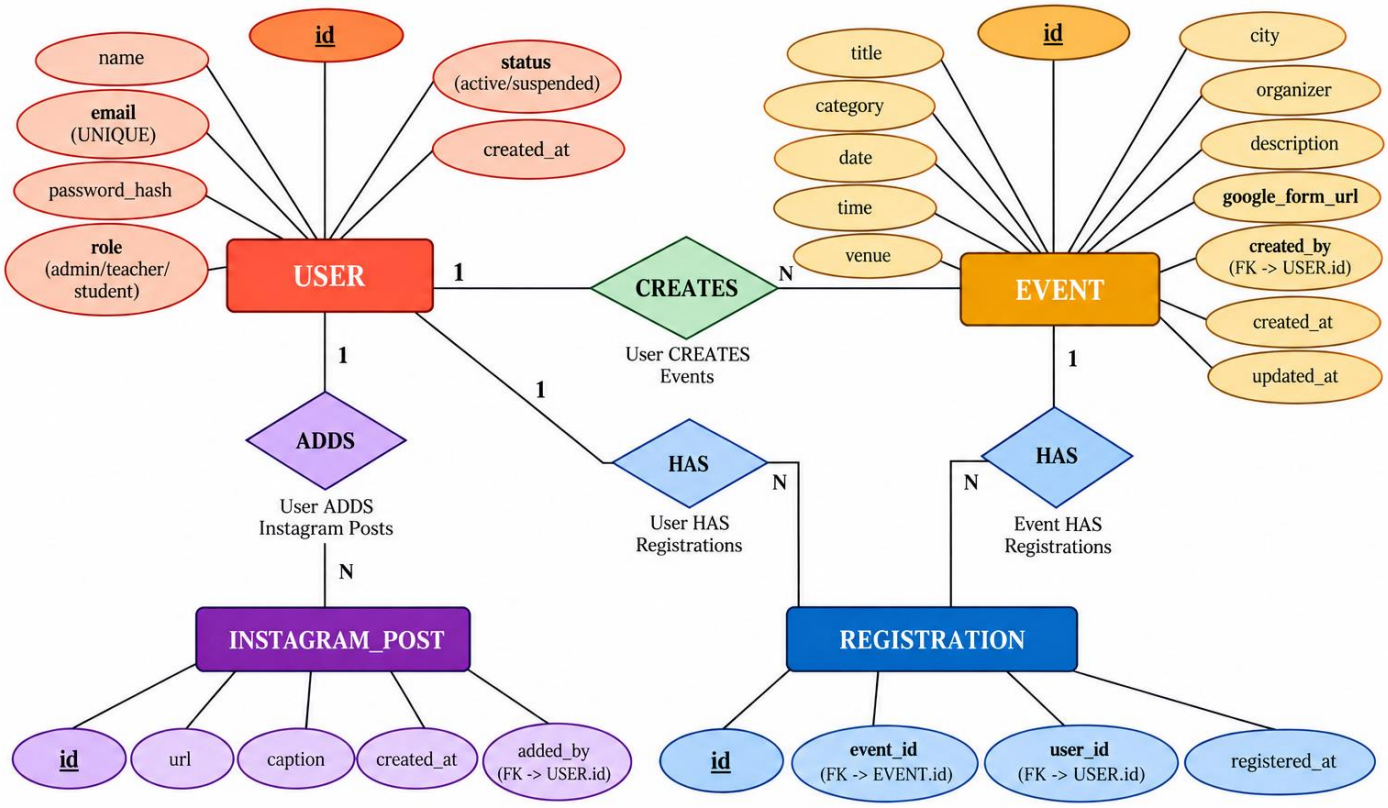
id	title	category	date	time	venue	city	organizer	created_by
1	Startup Founders Meetup	meetup	9/5/2026	18:30:00	WeWork Galaxy	Hyderabad	StartupSphere HYD	NULL
2	Hackathon 3.0 — 24 Hours	tech	9/15/2026	9:00:00	CS Block Lab 3, GEC	Ahmedabad	IEEE Student Chapter	NULL
3	Inter-Dept Cricket League	sports	9/25/2026	8:00:00	College Sports Ground	Ahmedabad	Sports Committee, GEC	NULL

3) **Table Name :** registrations

Primary Key: id (Auto Increment)

id	event_id	user_id	registered_at
1	6	2	10/3/2026 16:44
2	3	2	10/3/2026 16:48

➢ E-R Diagram:


id
name
email (UNIQUE)
password_hash
role (admin/teacher/ student)
status (active/suspended)
created_at
USER
1
CREATES
N
User CREATES Events
EVENT
id
title
category
date
time
venue
city
organizer
description
google_form_url
created_by (FK -> USER.id)
created_at
updated_at
1
ADDS
User ADDS Instagram Posts
N
1
HAS
N
User HAS Registrations
1
HAS
N
Event HAS Registrations
INSTAGRAM_POST
id
url
caption
created_at
added_by (FK -> USER.id)
REGISTRATION
id
event_id (FK -> EVENT.id)
user_id (FK -> USER.id)
registered_at

4.3 Data Dictionary

The data types are described as used by the code, because the SQL structure file was not part of the supplied project files.

1) USER:-

SR-NO	COLUMN NAME	DATATYPE	NULL	LINKS TO	COMMENTS
1	id	int(10) UNSIGNED	NO		Primary Key, Auto Increment
2	name	varchar(120)	NO		
3	email	varchar(190)	NO		Unique
4	password_hash	varchar(255)	NO		
5	role	enum('admin', 'teacher', 'student')	NO		Default 'student'
6	status	enum('active', 'suspended')	NO		Default 'active'
7	created_at	datetime	NO		Default current_timestamp()

2) events:

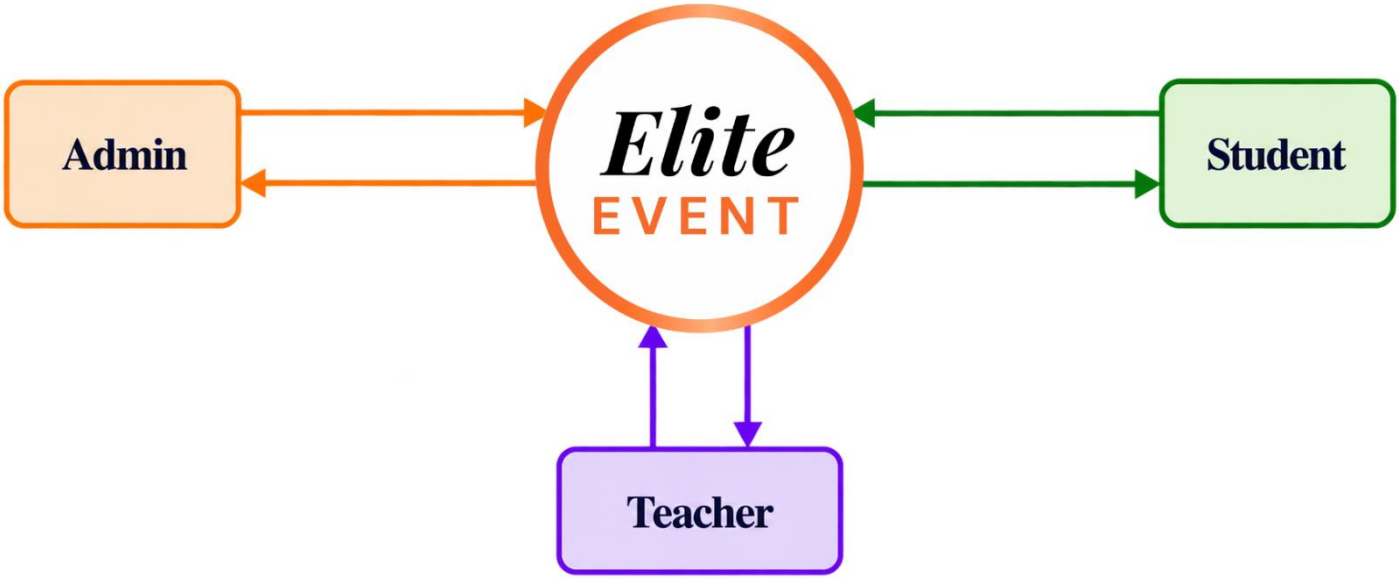
SR-NO	COLUMN NAME	DATATYPE	NULL	LINKS TO	COMMENTS
1	id	int(10) UNSIGNED	NO		Primary Key, Auto Increment
2	title	varchar(255)	NO		
3	category	varchar(60)	NO		
4	date	date	NO		
5	time	time	NO		
6	venue	varchar(255)	NO		
7	city	varchar(120)	NO		
8	organizer	varchar(190)	NO		
9	description	text	YES		
10	google_form_url	varchar(1000)	YES		
11	created_by	int(10) UNSIGNED	YES	users.id	Foreign Key, ON DELETE SET NULL
12	created_at	datetime	NO		Default current_timestamp()
13	updated_at	datetime	NO		Default current_timestamp(), ON UPDATE current_timestamp()

4)Registrations

SR-NO	COLUMN NAME	DATATYPE	NULL	LINKS TO	COMMENTS
1	id	int(10) UNSIGNED	NO		Primary Key, Auto Increment
2	event_id	int(10) UNSIGNED	NO	events.id	Foreign Key, ON DELETE CASCADE
3	user_id	int(10) UNSIGNED	NO	users.id	Foreign Key, ON DELETE CASCADE
4	registered_at	datetime	NO		Default current_timestamp()

4.3Functional and behavior modeling

4.3.1 Context Diagram(0-level Diagram)



4.3.2 First Level Diagram (1-level Diagram)

- **ADMIN :**



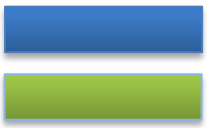
2) Teacher Side :



2) Student Side :



4.4 Gantt Chart



- Expected Time.
- Actual Time.

No.	Task	July	August	September	October
1	Requirement Gathering And SRS				
2	System Analysis				
3	System Design				
4	System Development				
5	System Testing				

5. TEST CASE

Login Module					
TEST CASE No. 1			TEST CASE DESIGN DATE : 16/Sep/2026		
TEST TITLE : Login Module Test			TEST CASE EXECUTION DATE : 19/Sep/2026		
DESCRIPTION : To test the student login page (student-login.php) and verify different login conditions					
PRECONDITION : Student login page must be available and valid user accounts must exist					
STEP No.	TEST STEP	TEST DATA	EXPECTED RESULT	ACTUAL RESULT	STATUS
1	Open the student login page	URL	Login form with e-mail and password is displayed.	Login form is displayed successfully.	PASS
2	Login with valid student account	Valid e-mail and password	User is redirected to dashboard.php.	User is redirected to dashboard.php successfully.	FAIL
3	Login with wrong password	Valid e-mail, wrong password	Error message is displayed and user stays on the login page.	Error message is displayed and user remains on the login page.	PASS
4	Login with a teacher account on student page	Teacher credentials	Login is refused because the role does not match.	Login is refused due to role mismatch.	PASS
5	Open login page while logged in	Active session	User is redirected to the dashboard according to the user's role.	User is redirected to the appropriate dashboard.	PASS

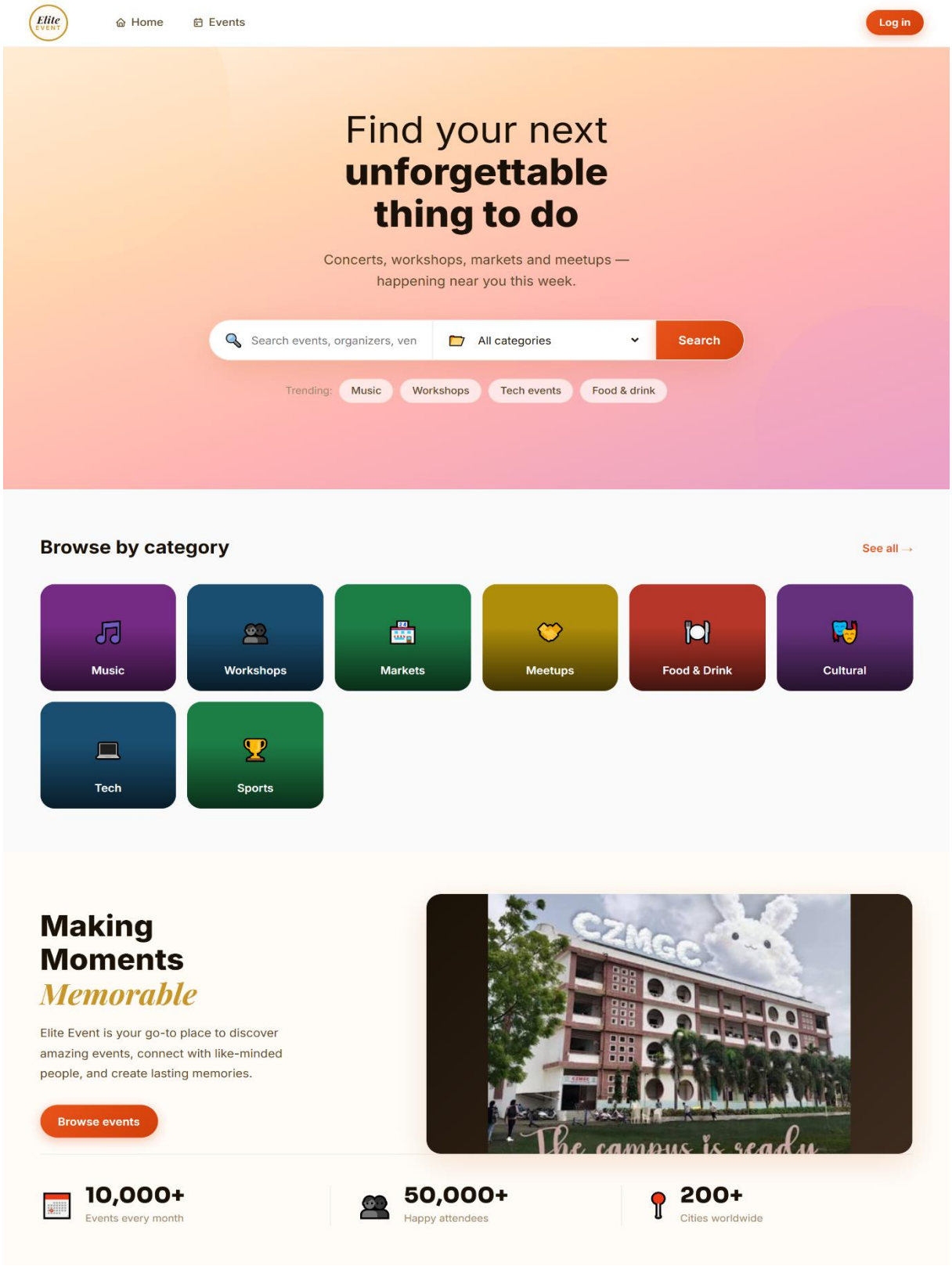
Event Registration Module					
TEST CASE No. 2			TEST CASE DESIGN DATE : 16/Sep/2026		
TEST TITLE : Event Registration Module			TEST CASE EXECUTION DATE : 16/Sep/2026		
DESCRIPTION : To test the event registration functionality in event-detail.php for different types of users and events.					
PRECONDITION : Event details must be available and the event registration module must be working.					
STEP No.	TEST STEP	TEST DATA	EXPECTED RESULT	ACTUAL RESULT	STATUS
1	Register without login	Not logged in	Panel shows **"Log in to register"**.	Panel shows "Log in to register".	PASS
2	Register as teacher	Teacher account	Message that only student accounts can register is displayed.	Message is displayed and registration is refused.	PASS
3	Register as student	Student account, event with form URL	Registration is saved and the Google Form is displayed.	Registration is saved and the Google Form is displayed.	PASS
4	Register again for the same event	Same student and event	Message that the student is already registered is displayed; no duplicate row is created.	Duplicate registration is prevented and the message is displayed.	PASS
5	Register for event without form URL	Event without Google Form	Message that registration is not available is displayed.	Registration is not allowed and the appropriate message is displayed.	PASS

Event Browsing and Search Module Test					
TEST CASE No. 3			TEST CASE DESIGN DATE : 16/Sep/2026		
TEST TITLE : Event Browsing and Search Module Test			TEST CASE EXECUTION DATE : 16/Sep/2026		
DESCRIPTION : To test event browsing, searching, and category filtering functionality in index.php and events.php.					
PRECONDITION : Home page and events page must be available with event data.					
STEP No.	TEST STEP	TEST DATA	EXPECTED RESULT	ACTUAL RESULT	STATUS
1	Open home page	URL	Category tiles, trending events, and Instagram section are displayed.	Category tiles, trending events, and Instagram section are displayed successfully.	PASS
2	Search by keyword	Keyword in the search box	Events matching the keyword are listed with the result count.	Matching events are displayed with the result count.	FAIL
3	Filter by category	Select a category	Only events of the selected category are listed.	Events of the selected category are displayed.	PASS
4	Search with no match	Unknown keyword	Message **"No events match your filters"** is displayed.	The message is displayed successfully.	PASS
5	Clear filters	Click **Clear all**	All events are listed in order of date.	All events are displayed in date order.	PASS

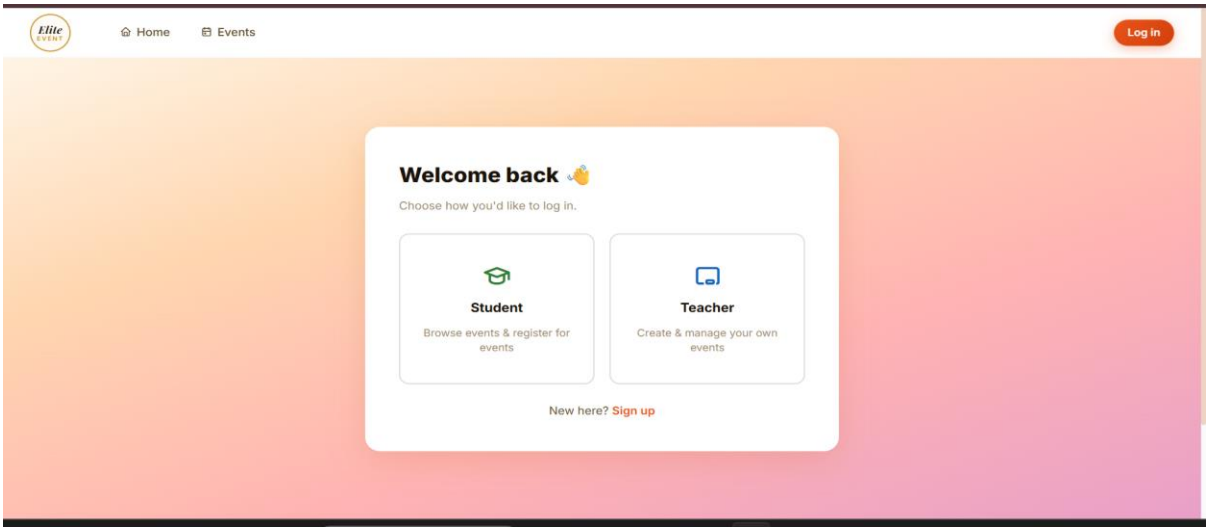
Event Registration Module					
TEST CASE No. 4			TEST CASE DESIGN DATE : 16/Sep/2026		
TEST TITLE : Event Registration Module			TEST CASE EXECUTION DATE : 16/Sep/2026		
DESCRIPTION : To test the event registration functionality in event-detail.php for different types of users and events.					
PRECONDITION : Event details must be available and the event registration module must be working.					
STEP No.	TEST STEP	TEST DATA	EXPECTED RESULT	ACTUAL RESULT	STATUS
1	Register without login	Not logged in	Panel shows **"Log in to register"**.	Panel shows **"Log in to register"** successfully.	PASS
2	Register as teacher	Teacher account	Message that only student accounts can register is displayed.	Message is displayed and registration is refused.	FAIL
3	Register as student	Student account, event with form URL	Registration is saved and the Google Form is displayed.	Registration is saved and the Google Form is displayed successfully.	PASS
4	Register again for the same event	Same student and event	Message that the student is already registered is displayed; no duplicate row is created.	Duplicate registration is prevented and the appropriate message is displayed.	PASS
5	Register for event without form URL	Event without Google Form	Message that registration is not available is displayed.	Registration is not allowed and the appropriate message is displayed.	PASS

6. SCREEN SHOTS
7.

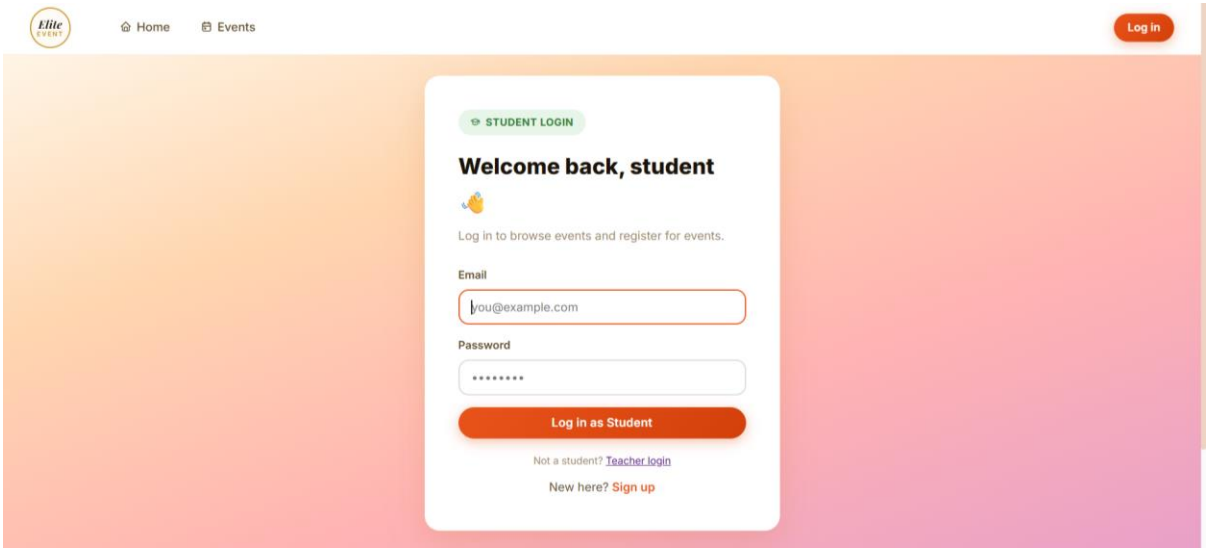
- **Home page**



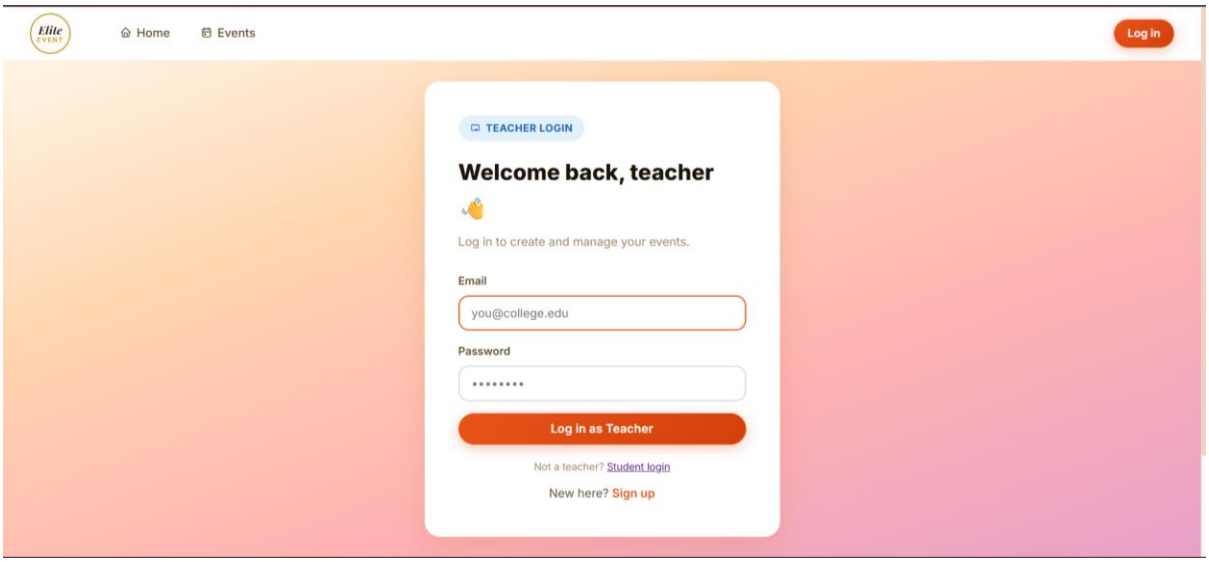
- **Login Page**



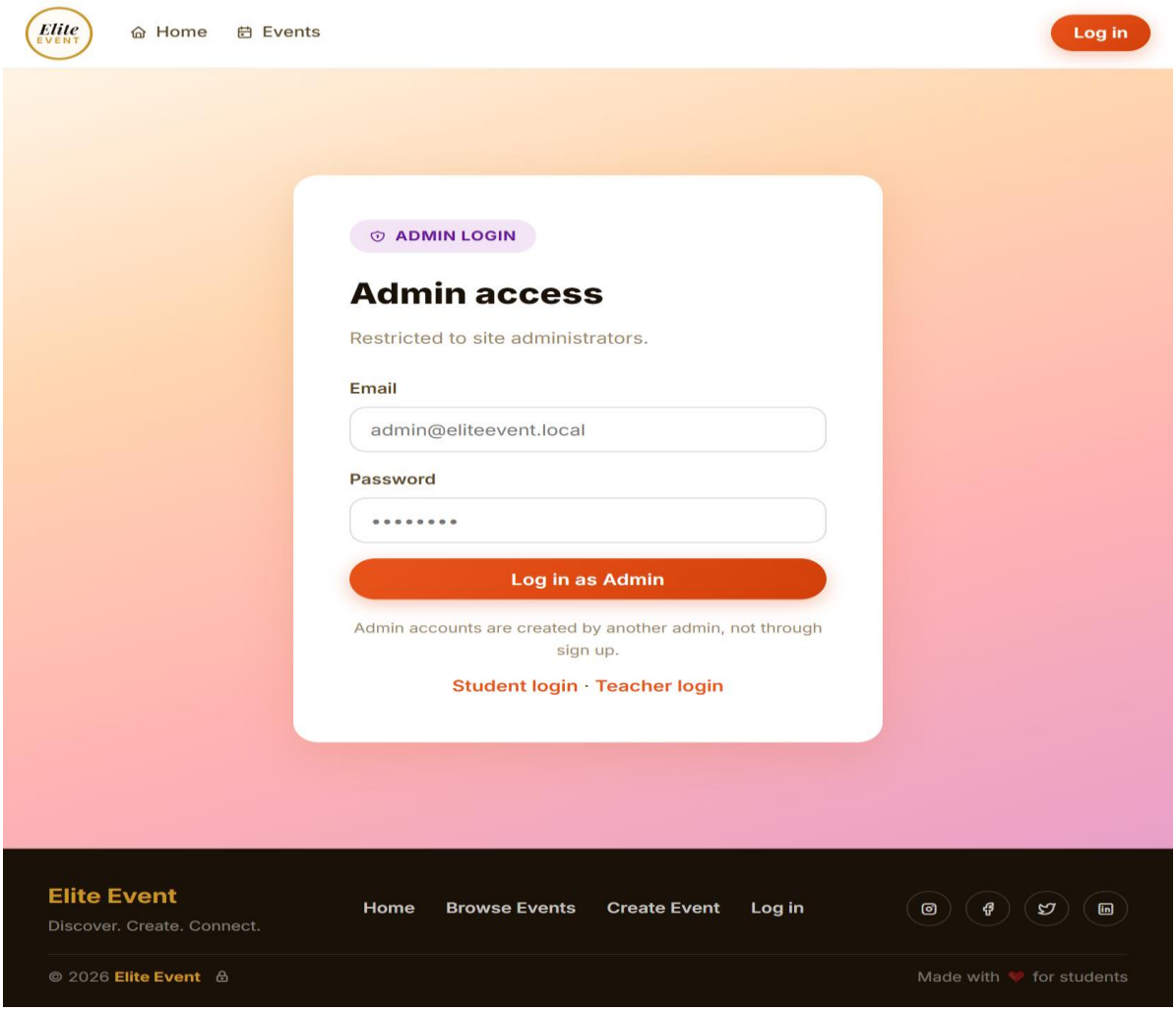
- **Student Login**



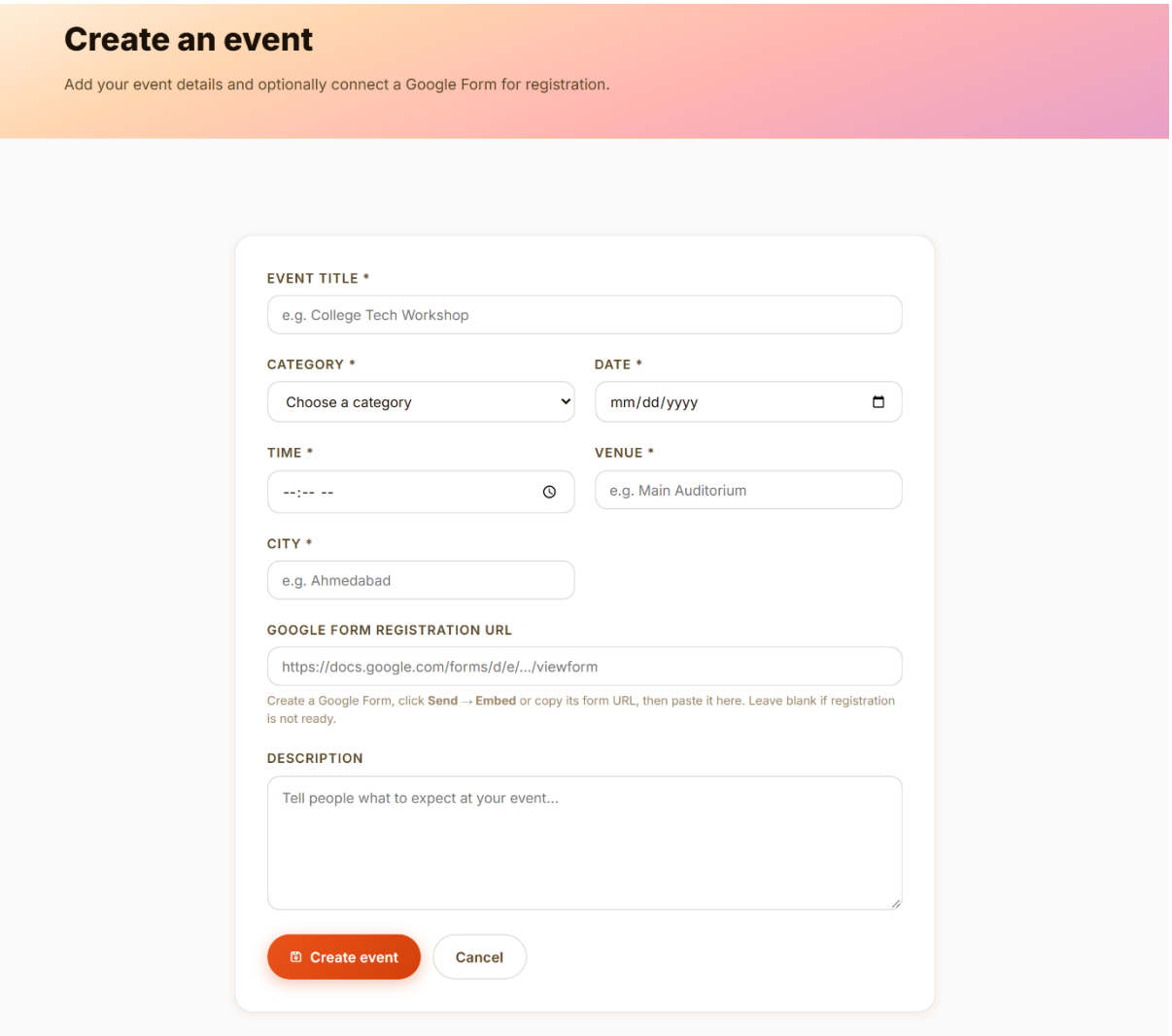
- **Teacher Login**



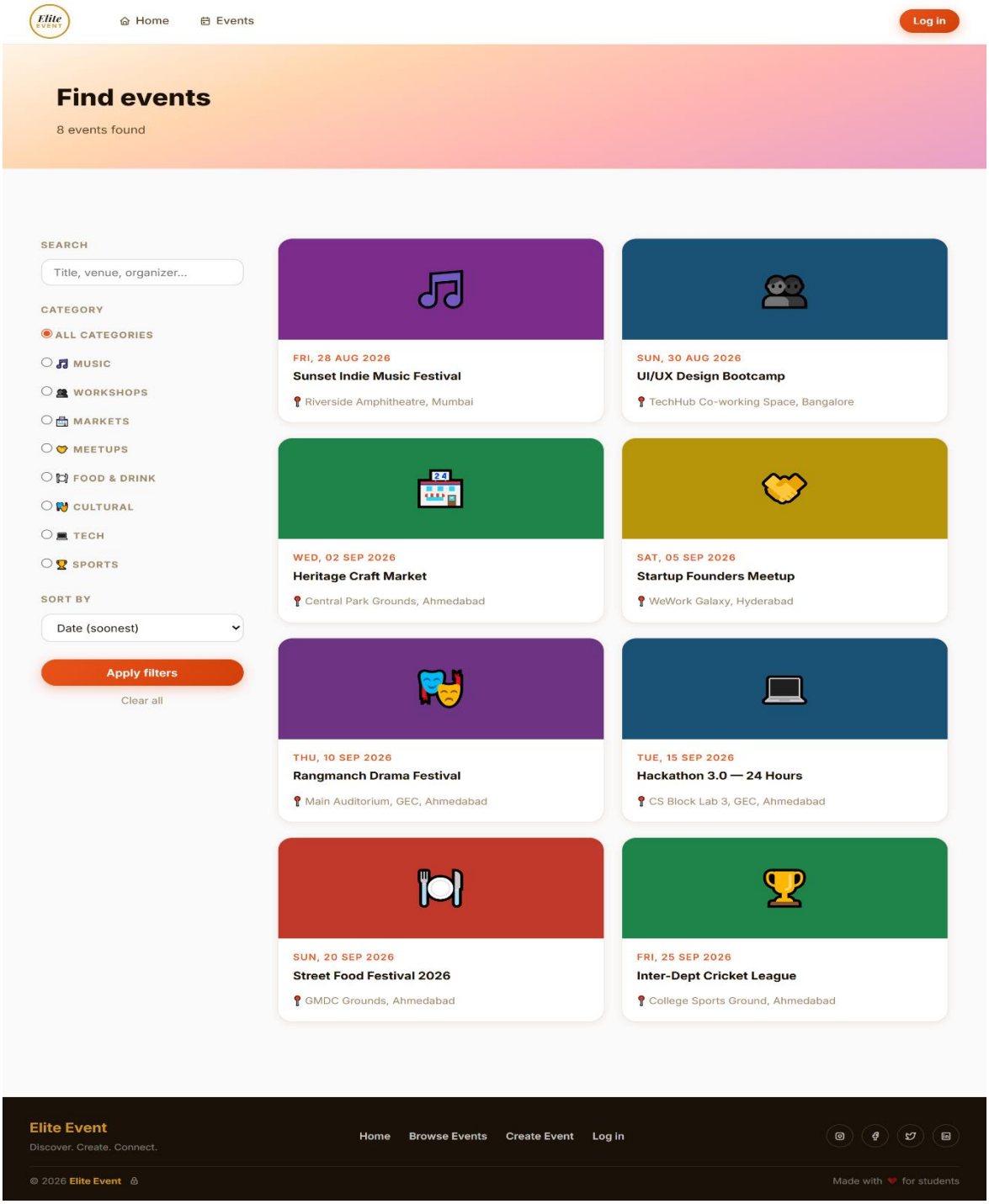
- **Admin Login**



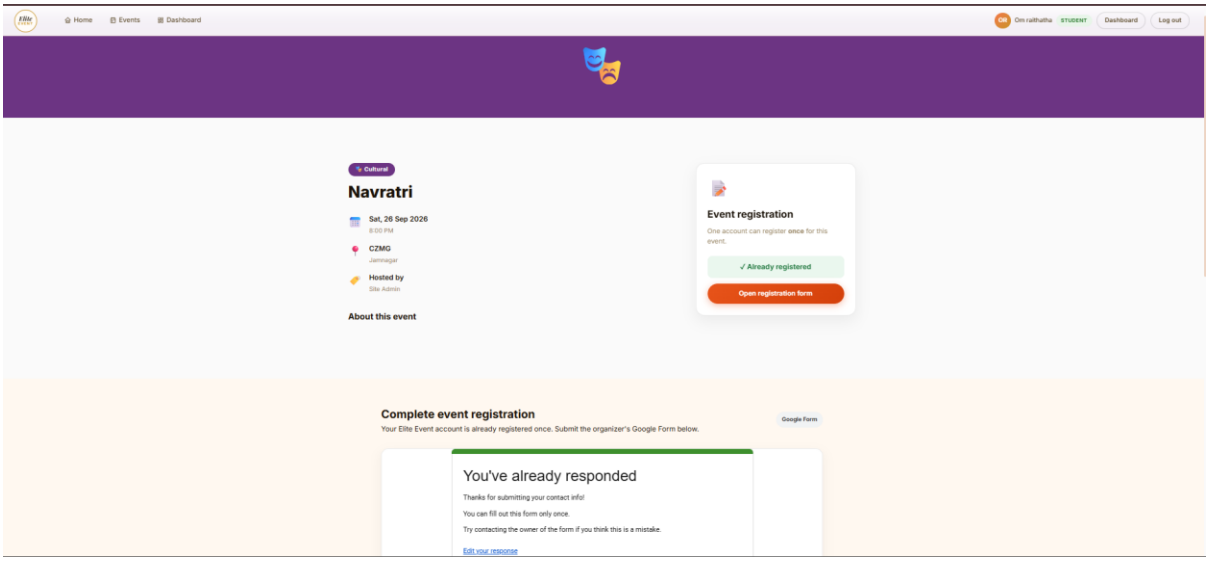
➢ Create Event



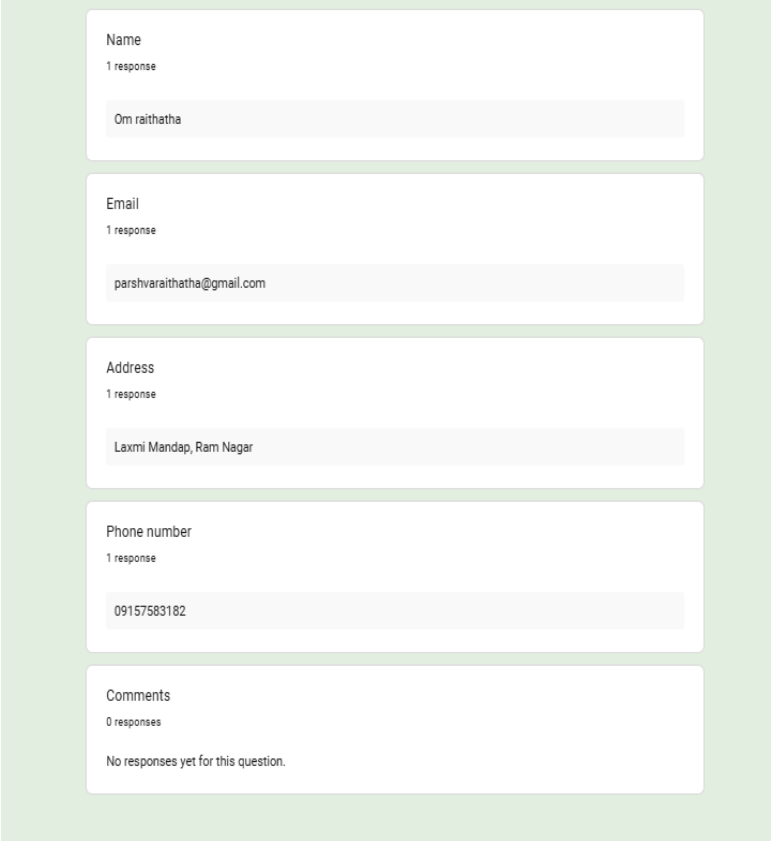
➢ **Find Event Page**



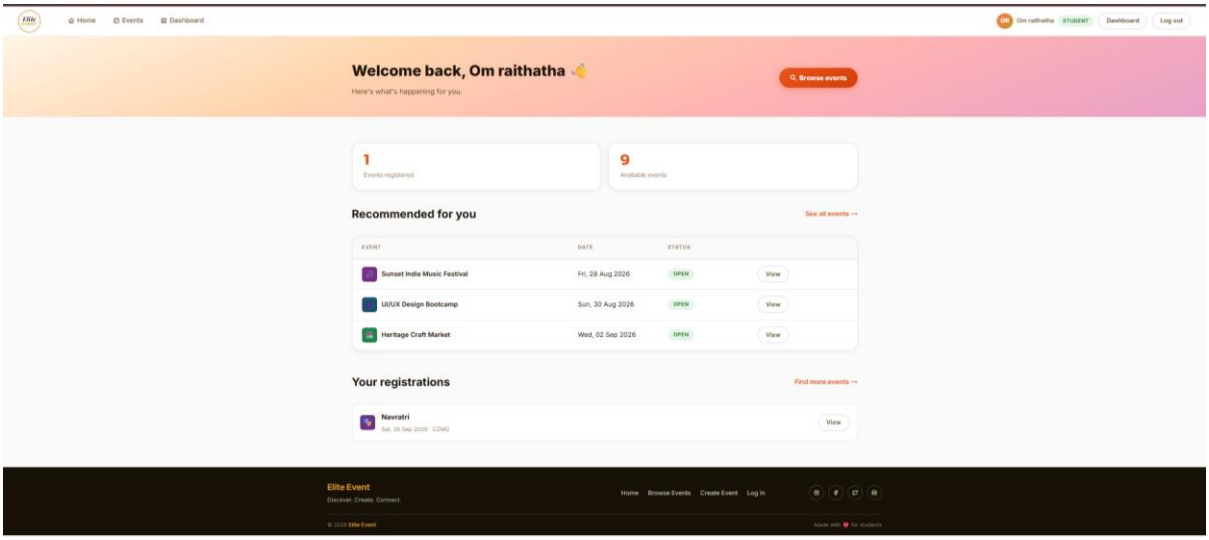
- **Event Detail Page**



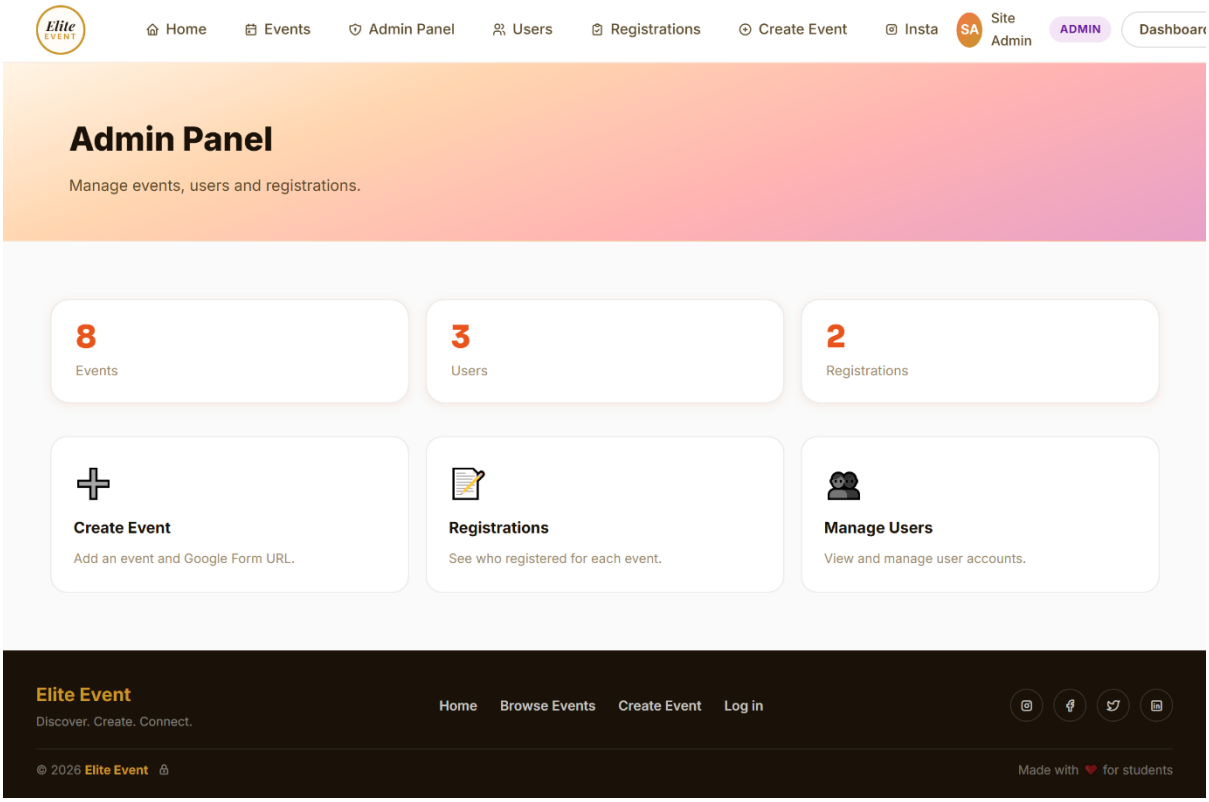
- **Google Form Registration**



- **Student Dashboard**



- **Admin Dashboard**



7. CODE EXPLANATION

1) Module Name: User Authentication

```php
function attempt_login(string $email, string $password, string $expectedRole) {
    $email = trim($email);
    if ($email === '' || $password === '') {
        return 'Enter your email and password.';
    }

    $stmt = db()->prepare('SELECT * FROM users WHERE email = ?');
    $stmt->execute([$email]);
    $user = $stmt->fetch();

    $genericError = 'Invalid email or password for this login type.';

    if (!$user || !password_verify($password, $user['password_hash'])) {
        return $genericError;
    }
    if ($user['role'] !== $expectedRole) {
        return $genericError;
    }
    if ($user['status'] !== 'active') {
        return 'This account has been suspended. Contact an administrator.';
    }

    session_regenerate_id(true);
    $_SESSION['user_id'] = $user['id'];

    return true;
}

function logout_user(): void {
    $_SESSION = [];
    if (ini_get('session.use_cookies')) {
        $params = session_get_cookie_params();
        setcookie(session_name(), '', time() - 42000, $params['path'], $params['domain'], $params['secure'], $params['httponly']);
    }
    session_destroy();
}
```

Purpose:

This module handles user authentication and logout functionality. It verifies the user's email and password, checks their assigned role and account status, and manages the login session.

Code Explanation:

- **attempt_login()** → Validates the login credentials provided by the user.
- **db()->prepare()** → Prepares the SQL query to retrieve the user's account.
- **password_verify()** → Checks the entered password against the stored password hash.
- **$expectedRole** → Ensures that the user logs in through the appropriate role-specific login page.
- **$user['status']** → Checks whether the account is active.

- **session_regenerate_id(true)** → Regenerates the session ID after successful authentication.
- **logout_user()** → Clears the session and destroys it during logout.

2) Module Name: Database Connection and Table Management

```php
function db(): PDO {
    static $pdo = null;
    if ($pdo !== null) return $pdo;
    $c = db_config();
    $charset = $c['charset'] ?? 'utf8mb4';
    try {
        $dsn = sprintf(
            'mysql:host=%s;port=%d;dbname=%s;charset=%s',
            $c['host'],
            (int)($c['port'] ?? 3306),
            $c['database'],
            $charset
        );

        $pdo = new PDO($dsn, $c['username'], $c['password'], [
            PDO::ATTR_ERRMODE => PDO::ERRMODE_EXCEPTION,
            PDO::ATTR_DEFAULT_FETCH_MODE => PDO::FETCH_ASSOC,
            PDO::ATTR_EMULATE_PREPARES => false,
        ]);
    } catch (PDOException $e) {
        if ((int)$e->getCode() === 1049) {
            $serverDsn = sprintf('mysql:host=%s;port=%d;charset=%s', $c['host'], (int)($c['port'] ?? 3306), $charset);
            $server = new PDO($serverDsn, $c['username'], $c['password'], [
                PDO::ATTR_ERRMODE => PDO::ERRMODE_EXCEPTION,
                PDO::ATTR_DEFAULT_FETCH_MODE => PDO::FETCH_ASSOC,
                PDO::ATTR_EMULATE_PREPARES => false,
            ]);
            $server->exec('CREATE DATABASE IF NOT EXISTS `' . str_replace('`', '``', $c['database']) . '` CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci');
            $server = null;
            $pdo = new PDO($dsn, $c['username'], $c['password'], [
                PDO::ATTR_ERRMODE => PDO::ERRMODE_EXCEPTION,
                PDO::ATTR_DEFAULT_FETCH_MODE => PDO::FETCH_ASSOC,
                PDO::ATTR_EMULATE_PREPARES => false,
            ]);
        } else {
            throw $e;
        }
    }
    db_install_or_migrate($pdo);
    return $pdo;
}

```

Purpose:

This module establishes a connection between the Elite Event application and its MySQL database. It provides a reusable database connection for event management, user authentication, and registration operations.

Code Explanation:

- **db()** → Provides the database connection used throughout the application.
- **PDO** → Provides the PHP interface for communicating with the database.
- **PDO::ATTR_ERRMODE** → Configures how database errors are handled.
- **PDO::FETCH_ASSOC** → Returns query results as associative arrays.
- **Database configuration** → Supplies the settings required to establish the connection.
- **Database exception handling** → Handles connection errors according to the configured error mode.

3) Module Name: Event Data Management

```php
$categories = [
    ['slug'=>'music','name'=>'Music','icon'=>'🎵','color'=>'#7B2D8B'],
    ['slug'=>'workshop','name'=>'Workshops','icon'=>'👩‍🎨','color'=>'#1A5276'],
    ['slug'=>'market','name'=>'Markets','icon'=>'🏪','color'=>'#1E8449'],
    ['slug'=>'meetup','name'=>'Meetups','icon'=>'🤝','color'=>'#B79508'],
    ['slug'=>'food','name'=>'Food & Drink','icon'=>'🍽️','color'=>'#C0392B'],
    ['slug'=>'cultural','name'=>'Cultural','icon'=>'🎭','color'=>'#6C3483'],
    ['slug'=>'tech','name'=>'Tech','icon'=>'💻','color'=>'#1A5276'],
    ['slug'=>'sports','name'=>'Sports','icon'=>'🏆','color'=>'#1E8449'],
];

function getAllEvents(): array {
    return db()->query('SELECT * FROM events ORDER BY date ASC, time ASC, id ASC')->fetchAll();
}

$events = getAllEvents();

function getEventById(array $events, int $id): ?array {
    foreach ($events as $e) if ((int)$e['id'] === $id) return $e;
    return null;
}

function getCategory(string $slug, array $categories): array {
    foreach ($categories as $cat) if ($cat['slug'] === $slug) return $cat;
    return ['slug'=>$slug,'name'=>ucfirst($slug),'icon'=>'📌','color'=>'#888'];
}

function formatEventDate(string $date): string {
    $ts = strtotime($date);
    return $ts ? date('D, d M Y', $ts) : $date;
}

function formatEventTime(string $time): string {
    $ts = strtotime($time);
    return $ts ? date('g:i A', $ts) : $time;
}

```

Purpose:

This module contains reusable functions and category information used to retrieve and display events throughout the application.

Code Explanation:

- **$categories** → Stores event categories and their associated display information.
- **getAllEvents()** → Retrieves events from the database.
- **getEventById()** → Finds an event using its ID within the supplied event array.
- **getCategory()** → Retrieves category information using a category slug.
- **formatEventDate()** → Formats an event date for display.
- **formatEventTime()** → Formats an event time for display.

4) Module Name: Student Dashboard



```php
<?php
require_once __DIR__ . '/includes/data.php';
$currentUser = require_role('student');

$pageTitle = 'Dashboard – Elite Event';
require_once __DIR__ . '/includes/header.php';

$registered = userRegistrations((int)$currentUser['id']);
$upcoming = array_slice($events, 0, 3);
$stats = [
    ['label' => 'Events registered', 'value' => count($registered)],
    ['label' => 'Available events',  'value' => count($events)],
];
?>
<section class="page-head">
  <div class="container dashboard-head">
    <div>
      <h1>Welcome back, <?= htmlspecialchars($currentUser['name']) ?> 👋</h1>
      <p>Here's what's happening for you.</p>
    </div>
    <a href="events.php" class="btn btn-primary"><i class="ti ti-search"></i> Browse events</a>
  </div>
</section>
<section class="section">
  <div class="container">

    <div class="stats-grid">
      <?php foreach ($stats as $s): ?>
        <div class="stat-card">
          <span class="stat-value"><?= htmlspecialchars($s['value']) ?></span>
          <span class="stat-label"><?= htmlspecialchars($s['label']) ?></span>
        </div>
      <?php endforeach; ?>
    </div>
    <div class="section-head">
      <h2>Recommended for you</h2>
      <a href="events.php" class="see-all">See all events →</a>
    </div>
```

Purpose:

This module provides students with a personalized dashboard showing registration statistics and available event information.

Code Explanation:

- **require_role('student')** → Restricts access to student accounts.
- **$currentUser** → Stores the currently logged-in student's information.
- **getAllEvents()** → Retrieves event information.
- **userRegistrations()** → Retrieves the student's registrations.
- **count($registered)** → Counts the student's registrations.
- **count($events)** → Counts the available event records.
- **$stats** → Stores statistics for display on the dashboard.
- **foreach** → Iterates through dashboard statistics to display them.

5) Module Name: Admin Dashboard

admin > dashboard.php > ...

```php
<?php
require_once __DIR__ . '/../includes/data.php';
$currentUser = require_role('admin');
$pageTitle = 'Admin Panel – Elite Event';
$userCount = (int)db()->query('SELECT COUNT(*) FROM users')->fetchColumn();
$registrationCount = (int)db()->query('SELECT COUNT(*) FROM registrations')->fetchColumn();
require_once __DIR__ . '/../includes/header.php';
?>
<section class="page-head"><div class="container"><h1>Admin Panel</h1><p>Manage events, users and registrations.</p></div></section>
<section class="section"><div class="container">
  <div class="stats-grid">
    <div class="stat-card"><span class="stat-value"><?= count($events) ?></span><span class="stat-label">Events</span></div>
    <div class="stat-card"><span class="stat-value"><?= $userCount ?></span><span class="stat-label">Users</span></div>
    <div class="stat-card"><span class="stat-value"><?= $registrationCount ?></span><span class="stat-label">Registrations</span></div>
  </div>
  <div class="admin-actions-grid">
    <a href="../create-event.php" class="admin-action-card"><span>➕</span><strong>Create Event</strong><small>Add an event and Google Form URL.</small></a>
    <a href="registrations.php" class="admin-action-card"><span>📝</span><strong>Registrations</strong><small>See who registered for each event.</small></a>
    <a href="users.php" class="admin-action-card"><span>👥</span><strong>Manage Users</strong><small>View and manage user accounts.</small></a>
  </div>
</div></section>
<?php require_once __DIR__ . '/../includes/footer.php'; ?>
```

Purpose:

This module provides administrators with an overview of the Elite Event system. It displays event, user, and registration statistics and provides access to administrative functions.

Code Explanation:

- **require_role('admin')** → Restricts dashboard access to administrators.
- **getAllEvents()** → Retrieves event records.
- **SELECT COUNT(*) FROM users** → Counts the user records in the database.
- **SELECT COUNT(*) FROM registrations** → Counts the registration records.
- **$userCount** → Stores the total number of users.
- **$registrationCount** → Stores the total number of registrations.
- **$events** → Contains the event records used by the dashboard.
- **Admin action cards** → Provide navigation to registration management and user management.

6) Module Name: Teacher Dashboard

```php
<?php
require_once __DIR__ . '/../includes/data.php';
$currentUser = require_role('teacher');
$pageTitle = 'Teacher Dashboard - Elite Event';
require_once __DIR__ . '/../includes/header.php';
?>
<section class="page-head"><div class="container"><h1>Teacher Dashboard</h1><p>Create and manage event listings.</p></div></section>
<section class="section"><div class="container">
  <div class="stats-grid"><div class="stat-card"><span class="stat-value"><?= count($events) ?></span><span class="stat-label">Published events</span></div></div>
  <div class="section-head"><h2>Your event tools</h2><a href="../create-event.php" class="btn btn-primary">Create Event</a></div>
  <div class="event-grid">
    <?php foreach ($events as $event): $cat=getCategory($event['category'],$categories); ?>
      <a class="event-card" href="../create-event.php?edit=<?= (int)$event['id'] ?>">
      <div class="event-card-cover" style="--cover-color:<?= htmlspecialchars($cat['color']) ?>"><span class="event-card-icon"><?= $cat['icon'] ?></span></div>
      <div class="event-card-body"><span class="event-card-date"><?= formatEventDate($event['date']) ?></span><h3><?= htmlspecialchars($event['title']) ?></h3>
      <p class="event-card-venue">/ Edit event</p></div></a>
    <?php endforeach; ?>
  </div>
</div></section>
<?php require_once __DIR__ . '/../includes/footer.php'; ?>
```

Purpose:

This module provides teachers with a dashboard for viewing their published events and accessing event management features.

Code Explanation:

- **require_role('teacher')** → Restricts dashboard access to teacher accounts.
- **$currentUser** → Stores the logged-in teacher's information.
- **$events** → Contains the events retrieved for the teacher dashboard.
- **count($events)** → Calculates the number of events displayed in the dashboard statistics.
- **getCategory()** → Retrieves category information for each event.
- **foreach** → Iterates through the events for display.
- **Event management links** → Provide access to the relevant event management pages.

8.LIMITATIONS AND FUTURE ENHANCEMENT

- **Limitations :-**

  - **External Storage:** Form responses stay in Google Forms/Sheets, not the database.
  - **Manual Setup:** Organizers must manually create forms; internet connection is required.
  - **No Capacity Control:** No seat limits, waitlists, or cancellation options.
  - **Unpersonalized Recommendations:** Shows only the first three events.
  - **Limited Sorting:** Filterable only by date.
  - **No Self-Registration:** All accounts must be created by administrators.
  - **No Alerts:** Lacks email and SMS notifications.
  - **Static Metrics:** Homepage stats are hardcoded text, not live data.

- **Future Enhancement:-**

  - **Capacity & Cancellation:** Add seat limits and registration cancellation.
  - **Automated Notifications:** Email/SMS confirmations and event reminders.
  - **Smart Recommendations:** Suggest events based on past registrations.
  - **Better Navigation:** Sort by title/category and add pagination.
  - **Native Database & Reports:** Store details internally and enable report exports.
  - **Online Payments:** Support payment gateways for paid events.
  - **Mobile & Multilingual:** Mobile app and multi-language support.
  - **UI/UX Upgrade:** Modernized web interface.

8. CONCLUSION

- **Purpose:** Centralizes college event discovery, student registration, and organizer management into one platform.
- **Core Features:**
    - **Students:** Browse events, log in, register via an integrated Google Form, and track statuses on a personal dashboard.
    - **Organizers:** Teachers and admins create, edit, and link Google Forms to events.
- **Tech Stack:** PHP (backend with PDO), relational database, HTML5/CSS3/JavaScript (frontend), plus Google Forms and Instagram embeds.
- **Security:** Role-based access control, CSRF tokens, prepared statements, output escaping, and Google Form URL validation.

9. REFERENCES AND BIBLIOGRAPHY

The following official documentation and resources are relevant to the technologies used in the ELITE EVENT system.

Technologies and Libraries:

- PHP
- PDO (PHP Data Objects)
- HTML5
- CSS3
- JavaScript
- MySQL
- XAMPP
- Google Forms
- Instagram embeds

Websites:

- PHP Manual – https://www.php.net/manual/en/
- MySQL Reference Manual – https://dev.mysql.com/doc/
- MDN Web Docs (HTML, CSS, JavaScript) – https://developer.mozilla.org/
- XAMPP – https://www.apachefriends.org/
- Tabler Icons – https://tabler.io/icons
- Google Docs Editors Help (Google Forms) – https://support.google.com/docs/
- Graphviz documentation (used to draw the diagrams in this report) – https://graphviz.org/documentation/